

Linux 장애 분석 과제 2 수행 보고서

AI/SW Basic · Linux와 OS · 2026-09-18

교육기관 제공 agent-app-leak.zip의 Linux x86 실행 파일로 OOM Crash, CPU Latency, Deadlock을 재현했다. 각 사례는 발생 현상, 원문 로그·관제 증거, 원인 분석, 환경변수 변경 전후 검증의 네 항목으로 작성했다. 보너스 스케줄링 분석과 공통 Issue 템플릿을 덧붙였다.

주요 결과

사례	변경	관측 결과
OOM	MEMORY_LIMIT 64 → 128 MB	MemoryGuard 종료까지 8.418 → 17.536초, 약 2.08배 생존
CPU	CPU_MAX_OCCUPY 100 → 40%	27.302초 Watchdog 종료 → 90초 이상 생존
Deadlock	MULTI_THREAD_ENABLE true → false	순환 락 대기·79.317초 로그 정체 → 90초간 작업 진행
보너스	정상 시나리오 작업 로그	A→B→C 교대·진행률 재개: 앱 수준 Round-Robin 패턴

실행 환경과 증거

WSL2 Linux 6.6.87.2, x86_64, 논리 CPU 12개, 일반 계정 kju(UID 1000). 사용자·네트워크 네임스페이스를 분리해 기존 서비스와 충돌 없이 0.0.0.0:15034에 바인딩했다. CPU·메모리는 호스트와 공유한다. 실제 앱의 부팅 검사 6단계가 모두 통과했다.

앱 원본 SHA256:

```
7e0a19cfa80ece6b547a5008273661f0d4d71e526e96b51e0d0f341dd1bb3e40
```

OOM·Deadlock은 0.5초, CPU는 0.1초 간격으로 관측했다. 각 비교 쌍에서는 변수 하나만 바꾸고 90초 관찰 상한을 동일하게 적용했다. 앱 로그 시각은 KST(UTC+9), CSV와 수집기 로그는 UTC다. 그래프는 정확한 소켓 소유 워커의 CSV에서 생성했다.

수집기 테스트 11개, 실제 원본·부팅·비교 조건·장애 증거 검사 50개가 통과했다. 실행 명령과 증거 경로는 evidence/manifest.json, 계산 결과는 evidence/comparison.json에 있다.

첨부 자료 안내

이 PDF에는 source-and-evidence.zip을 첨부했다. 첨부를 지원하는 PDF 뷰어에서 원본 CSV·로그·ps/top 출력·수집 코드와 SHA256 목록을 열 수 있다. 같은 ZIP을 PDF 옆에도 별도로 제공한다. 교육기관 제공 바이너리와 실행용 키 파일은 ZIP에 넣지 않았다. 보고서의 상대 링크는 함께 제공한 폴더 구조에서 사용할 수 있다.

결과 해석의 범위

메모리 상한 증가는 누수 자체의 근본 해결이 아니고, 90초 생존은 장기간 안정성 보장이 아니다. OOM은 앱 MemoryGuard 종료로 확인했으며 커널 OOM으로 단정하지 않았다. 앱 내부 Current Load와 OS CPU를 구분했고, 실제 사용자 요청 지연은 측정하지 않았다. 모든 분석은 원본 앱 실행 로그와 Linux 관제 결과를 사용했으며 바이너리 역공학은 수행하지 않았다.

[Bug] OOM Crash - 메모리 누적 후 MemoryGuard가 SIGKILL로 종료

실측 완료 · 2026-09-18 · 교육기관 제공 agent-leak-app-x86

1. Description (현상 설명)

WSL2 Linux x86_64에서 일반 계정 `kjw`(UID 1000)로 실행했다. 부팅 검사 6개와 `Agent READY`를 통과한 뒤 `MemoryWorker`의 메모리가 약 3초마다 25 MB씩 증가했다. `MEMORY_LIMIT=64`에서 실행 후 8.418초에 `MemoryGuard`가 강제 종료했다. 한도를 128로 높인 실행은 17.536초까지 생존했지만 같은 원인으로 종료했다.

두 실행에서 `CPU_MAX_OCCUPY=100`, `MULTI_THREAD_ENABLE=false`를 고정했다. 독립된 사용자·네트워크 네임스페이스에서 UID 1000으로 `0.0.0.0:15034`에 정상 바인딩했다. 바이너리는 수정하지 않았다. [실험 목록](#), [원본 SHA256](#).

2. Evidence & Logs (증거 자료)

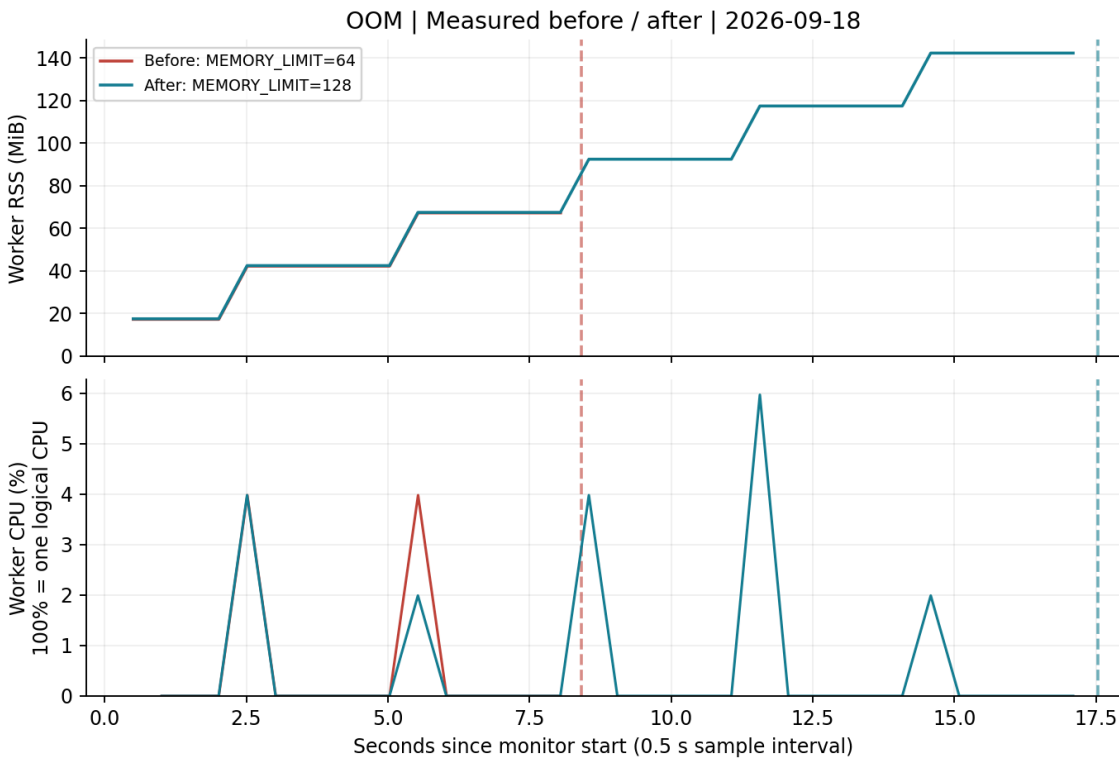
재현 명령

관찰 상한은 90초, 모니터 간격은 0.5초다. 저장소 루트에서 일반 사용자로 실행한다.

```
unshare --user --map-current-user --net bash scripts/run-case.sh \
--app vendor/agent-leak-app-x86 --case oom-before \
--duration 90 --interval 0.5 --snapshot-interval 5
unshare --user --map-current-user --net bash scripts/run-case.sh \
--app vendor/agent-leak-app-x86 --case oom-after \
--duration 90 --interval 0.5 --snapshot-interval 5
```

소켓 소유 워커는 Before 2440463, After 2440969였다. 약 2 MiB인 패키징 실행기 PID와 구분했다.

실행	워커 RSS 첫 샘플 → 마지막/최대	앱 로그 Heap 진행	결과
Before	17.250 → 67.250 MiB	25 → 50 → 75 MB	75 ≥ 64, 보호 종료
After	17.484 → 142.359 MiB	25 → 50 → ... → 150 MB	150 ≥ 128, 보호 종료



Before의 원문 발췌. 앱 시각은 KST(UTC+9)이며 관제 CSV와 수집기 시각은 UTC다.

```
2026-09-18 18:06:16,803 [INFO] [MemoryWorker] Current Heap: 25MB
2026-09-18 18:06:19,834 [INFO] [MemoryWorker] Current Heap: 50MB
2026-09-18 18:06:22,866 [INFO] [MemoryWorker] Current Heap: 75MB
2026-09-18 18:06:22,867 [CRITICAL] [MemoryGuard] Memory limit exceeded (75MB >= 64MB)
/ (Recommend Over 256MB)
2026-09-18 18:06:22,868 [CRITICAL] [MemoryGuard] Self-terminating process 2440463 to
prevent system instability.
```

After의 종료 직전 원문:

```
2026-09-18 18:06:57,505 [INFO] [MemoryWorker] Current Heap: 125MB
2026-09-18 18:07:00,533 [INFO] [MemoryWorker] Current Heap: 150MB
2026-09-18 18:07:00,534 [CRITICAL] [MemoryGuard] Memory limit exceeded (150MB >=
128MB) / (Recommend Over 256MB)
2026-09-18 18:07:00,535 [CRITICAL] [MemoryGuard] Self-terminating process 2440969 to
prevent system instability.
```

두 실행의 result.json은 reason=app_exited, alive_before_cleanup=false, launcher_returncode=-9, runner_events=[]다. 수집기가 종료 신호를 보내기 전에 앱 그룹이 종료됐고, 실행기는 SIGKILL 종료 상태를 반환했다. 종료 후 monitor.log에는 EVENT:PROCESS_EXITED가 기록됐다.

원문 자료:

- Before: [관제 로그](#), [CSV](#), [실행 로그](#), [ps-top-ss](#), [종료 결과](#)
- After: [관제 로그](#), [CSV](#), [실행 로그](#), [종료 결과](#)

3. Root Cause Analysis (원인 분석)

관측된 직접 원인은 메모리 누적과 앱의 MemoryGuard 임계치 초과다. Heap 출력과 RSS가 함께 증가하고, Guard가 초과를 기록한 직후 종료됐다. 할당된 데이터를 종료 전까지 충분히 회수하지 않는 누수성 패턴과 일치한다. 다만 제공 바이너리를 분석하지 않았으므로 어떤 객체·참조가 원인인지, 의도된 실습용 누적과 실제 결함을 내부 구현 수준에서 구분할 수는 없다.

이 결과는 커널 OOM killer의 시스템 메모리 고갈 판정이 아니다. 앱이 자체 보호 종료를 명시했고 수집기 개입도 없었다. 커널 OOM 로그나 시스템 전체 메모리 고갈은 입증하지 않았다.

Guard의 Heap MB와 OS의 RSS MiB는 같은 지표가 아니다. RSS에는 런타임과 공유 라이브러리도 포함된다. 0.5초 샘플 사이에 마지막 할당과 종료가 연속해서 발생하므로 마지막 RSS 샘플은 종료 순간을 놓칠 수 있다. 따라서 로그의 75/150 MB와 RSS 최대 67.250/142.359 MiB를 같은 값으로 취급하지 않는다.

4. Workaround & Verification (조치 및 검증)

항목	Before	After
MEMORY_LIMIT	64 MB	128 MB
고정 변수	CPU=100%, 멀티스레드=false	동일
관찰 상한	90초	90초
실제 생존 시간	8.418초	17.536초
종료 상태	MemoryGuard / -9	MemoryGuard / -9
최대 RSS	67.250 MiB	142.359 MiB

한도 상향으로 생존 시간이 9.118초 증가, 약 2.08배가 됐다. 메모리 증가 자체는 계속됐으므로 임시 완화다. 이후 MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=false의 정상 관측에서는 앱의 캐시 정리와 계속되는 작업 로그를 확인했다. 이 추가 관측은 다른 변수도 바뀌었으므로 위 단일 변수 비교와 분리해 해석한다.

근본 개선 제안은 불필요한 참조 제거, 캐시·큐 크기 제한, 주기적인 회수, 장시간 부하 검증이다. 바이너리의 소스 수정은 수행하지 않았다.

[Bug] CPU Latency - CPU 급상승과 Watchdog의 SIGTERM 보호 종료

실측 완료 · 2026-09-18 · 교육기관 제공 agent-leak-app-x86

1. Description (현상 설명)

MEMORY_LIMIT=512, MULTI_THREAD_ENABLE=false를 고정하고 CPU 설정만 비교했다. CPU_MAX_OCCUPY=100에서 워커의 짧은 CPU 상승 구간을 관측했고, 앱 내부 부하 값이 52.69%가 되자 Watchdog가 종료했다. 실행부터 종료 관측까지 27.302초였다. CPU_MAX_OCCUPY=40으로 낮춘 실행은 90초 이상 생존하며 부하 증가·냉각과 작업 로그를 계속 남겼다.

공통 환경은 WSL2 Linux x86_64, 논리 CPU 12개, UID 1000이다. 독립 네트워크에서 앱이 0.0.0.0:15034에 정상 바인딩했다. 소켓 소유 워커 PID는 Before 2445628, After 2447020이다. HTTP 응답시간이나 실제 사용자 요청 지연은 측정하지 않았으므로 지연 감소를 실측 성과로 주장하지 않는다.

2. Evidence & Logs (증거 자료)

재현 명령

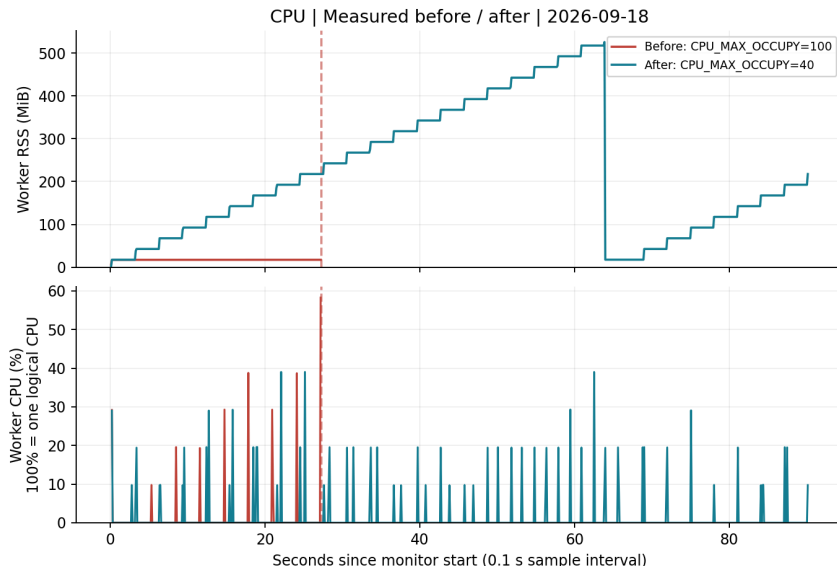
짧은 CPU 상승을 포착하기 위해 양쪽 모두 0.1초 간격으로 관측했다.

```
unshare --user --map-current-user --net bash scripts/run-case.sh \
--app vendor/agent-leak-app-x86 --case cpu-before \
--duration 90 --interval 0.1 --snapshot-interval 5
unshare --user --map-current-user --net bash scripts/run-case.sh \
--app vendor/agent-leak-app-x86 --case cpu-after \
--duration 90 --interval 0.1 --snapshot-interval 5
```

CPU 상승과 종료 원문

Before의 metrics.csv에서 동일 워커의 CPU 구간 값이 다음과 같이 변했다. CPU 100%는 논리 CPU 한 개이며, 최초 샘플은 계산 구간이 없어 비워 둔다.

UTC 시각	워커 PID	구간 CPU	RSS KiB	누적 CPU ticks
09:11:48.850	2445628	0.00%	17792	23
09:11:48.954	2445628	0.00%	17792	23
09:11:49.057	2445628	58.26%	17792	29



앱 로그 원문은 KST(UTC+9)다. 앱의 Current Load는 OS 관제 CPU와 별도 지표이며 동일 값으로 취급하지 않는다.

```
2026-09-18 18:11:45,871 [INFO] [CpuWorker] Current Load: 44.69%
2026-09-18 18:11:48,987 [INFO] [CpuWorker] Current Load: 52.69%
2026-09-18 18:11:49,088 [CRITICAL] [CpuWorker] CPU Threshold Violated! (52.69%).

>>> [SYSTEM] WATCHDOG: INITIATING EMERGENCY ABORT (SIGTERM) <<<
```

Before 결과는 `app_exited, launcher_returncode=-15, runner_events=[]` 다. 앱이 명시한 Watchdog 신호와 실제 SIGTERM 종료 상태가 일치하며, 수집기는 종료 신호를 보내지 않았다.

After에서는 다음과 같이 정상적인 냉각과 재증가가 이어졌다.

```
2026-09-18 18:13:51,956 [INFO] [CpuWorker] Peak reached (40.00%). Starting cooldown...
2026-09-18 18:13:52,963 [INFO] [CpuWorker] Current Load: 40.00%
2026-09-18 18:14:07,533 [INFO] [CpuWorker] Cooldown complete (5.00%). Resuming load
increase...
2026-09-18 18:14:08,539 [INFO] [CpuWorker] Current Load: 5.00%
2026-09-18 18:14:17,883 [INFO] [CpuWorker] Current Load: 19.12%
```

원문 자료:

- Before: [관제 로그](#), [CSV](#), [실행 로그](#), [ps-top-ss](#), [종료 결과](#)
- After: [관제 로그](#), [CSV](#), [실행 로그](#), [종료 결과](#)

3. Root Cause Analysis (원인 분석)

직접 원인은 앱 내부 부하 증가와 Watchdog 정책 위반이다. 본 바이너리는 CPU 설정 100%에서 부하를 높이다가 내부 값 52.69%에서 보호 종료했고, 40%에서는 고점 도달 후 냉각했다. 따라서 CPU_MAX_OCCUPY를 보호 임계값 그 자체로 해석해 높이는 조치는 이 앱에 맞지 않는다. 로그상 최대 부하 설정을 낮추는 방향으로 회피했다. 정확한 내부 측정·분기 코드는 분석하지 않았다.

관측한 OS CPU 급상승은 0.1초 구간의 58.26%다. 시스템 전체가 장시간 100% 과부하였다는 뜻은 아니다. 초기 0.5초 관측에서는 짧은 부하가 평균에 희석됐으므로 더 짧은 동일 간격의 최종 비교를 사용했다. `ps %CPU`는 수명 평균이라 해당 순간의 고점을 대신할 수 없다.

CPU 연산이 길어지면 실행 대기 중인 다른 작업의 응답이 늦어질 수 있다. 이 실험은 특정 프로세스의 상승과 앱 보호 정책의 종료를 입증하며, 외부 서비스 지연이나 커널 장애는 입증하지 않는다. After에서 정상 시나리오로 전환되어 메모리 워커도 함께 동작했으므로 CPU 지표 차이를 동일 연산량의 성능 개선율로 계산하지 않았다.

4. Workaround & Verification (조치 및 검증)

항목	Before	After
CPU_MAX_OCCUPY	100%	40%
고정 변수	MEMORY_LIMIT=512, 멀티스레드=false	동일
관찰 상한 / 샘플 간격	90초 / 0.1초	동일
OS 최대 구간 CPU	58.26%	39.02%
앱 Current Load	52.69%에서 정책 위반	40.00% 고점 후 냉각
생존	27.302초 후 Watchdog 종료	90초 이상 생존
최종 -15의 주체	앱 Watchdog	관찰 종료 후 수집기 정리

After의 `result.json`은 `observation_timeout, alive_before_cleanup=true, observed_s=90.020`이고 수집기 SIGTERM 시각이 별도로 기록됐다. 따라서 After의 -15를 Watchdog 재발로 오인하지 않는다.

실습용 임시 설정은 CPU_MAX_OCCUPY=40이다. 근본 개선은 불필요한 반복 연산-busy wait 점검, 작업 분할, 워커 수·요청량 제어 및 실제 응답시간 검증이다. 이 관측만으로 장기간 무장애를 보장하지 않는다.

[Bug] Deadlock - 두 워커의 순환 락 대기로 작업 진행 중단

실측 완료 · 2026-09-18 · 교육기관 제공 agent-leak-app-x86

1. Description (현상 설명)

MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40을 고정하고 멀티스레드 설정만 비교했다. true에서 실행 약 9초 뒤 두 워커가 서로의 자원을 기다리는 로그를 남겼다. 이후 PID는 계속 존재했지만 CPU-RSS와 로그 크기가 정체됐다. false로 변경한 실행은 동일한 90초 동안 작업·메모리 회수·CPU 냉각 로그가 계속 진행됐다.

일반 계정 UID 1000, WSL2 x86_64, 독립 네트워크의 고정 포트 15034에서 관측했다. 워커 PID는 Before 2448740, After 2451259다. 종료 여부뿐 아니라 작업의 진행 여부를 함께 확인했다.

2. Evidence & Logs (증거 자료)

재현 명령

```
unshare --user --map-current-user --net bash scripts/run-case.sh \
--app vendor/agent-leak-app-x86 --case deadlock-before \
--duration 90 --interval 0.5 --snapshot-interval 5
unshare --user --map-current-user --net bash scripts/run-case.sh \
--app vendor/agent-leak-app-x86 --case deadlock-after \
--duration 90 --interval 0.5 --snapshot-interval 5
```

마지막 진행 로그와 순환 대기

Before의 실제 로그 발췌. 시각은 KST다.

```
2026-09-18 18:14:40,866 [INFO] [AgentWorker][Worker-Thread-1] LOCK ACQUIRED:
[Shared_Memory_A]. (Holding...)
2026-09-18 18:14:40,866 [INFO] [AgentWorker][Worker-Thread-2] LOCK ACQUIRED:
[Socket_Pool_B]. (Holding...)
2026-09-18 18:14:42,870 [INFO] [AgentWorker][Worker-Thread-1] Need resource
[Socket_Pool_B] to finish job.
2026-09-18 18:14:42,871 [INFO] [AgentWorker][Worker-Thread-1] WAITING for
[Socket_Pool_B]... (Status: BLOCKED)
2026-09-18 18:14:42,879 [INFO] [AgentWorker][Worker-Thread-2] Need resource
[Shared_Memory_A] to write logs.
2026-09-18 18:14:42,880 [INFO] [AgentWorker][Worker-Thread-2] WAITING for
[Shared_Memory_A]... (Status: BLOCKED)
```

PID·스레드·자원 정체

09:16:03.609 UTC의 ps -L 원문 발췌다. 동일 PID/TID들이 앞선 09:14:44.299 UTC에도 존재했다. 실행기 PID 2448732는 별도로 do_wait 상태였고, 실제 워커는 다음 세 스레드였다.

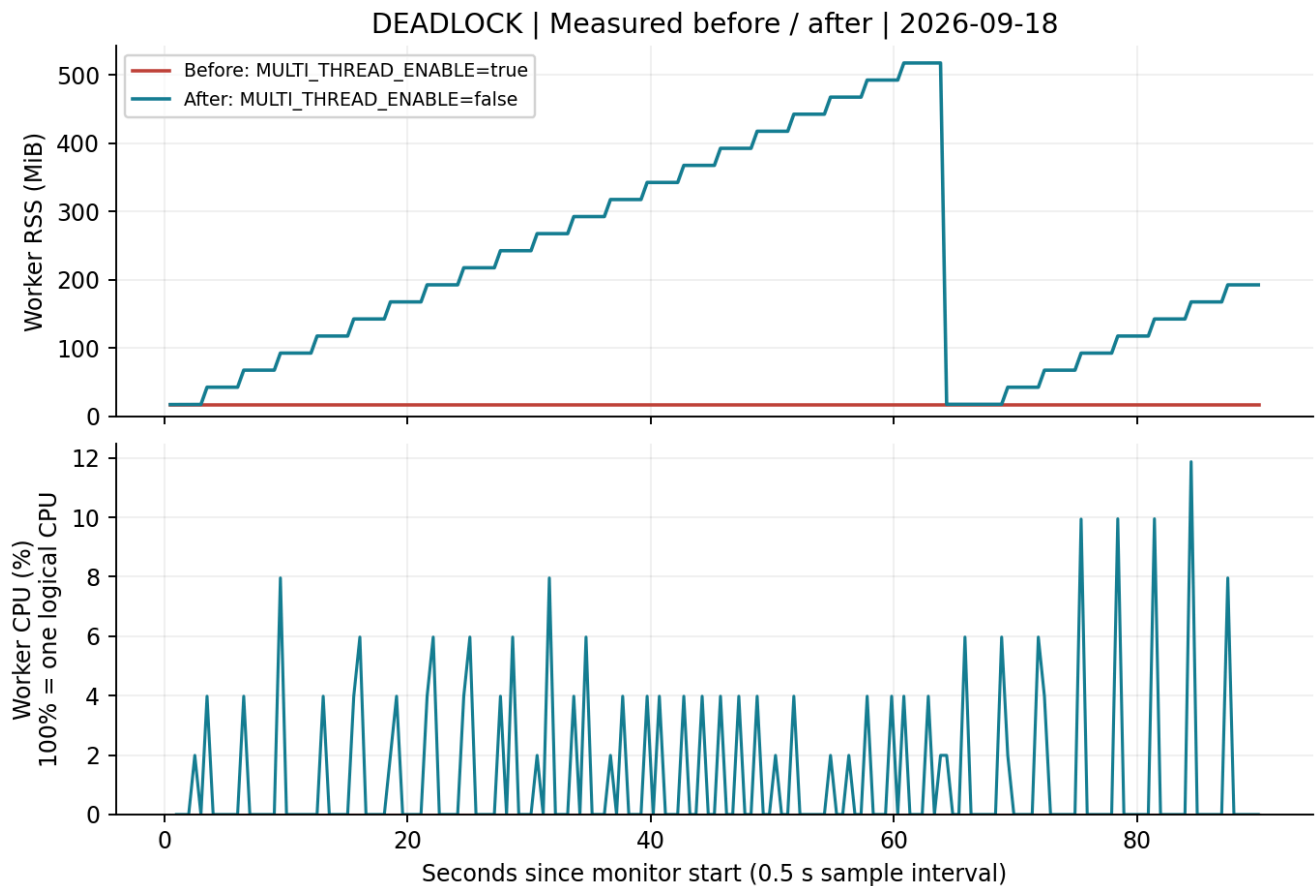
PID	TID	STAT	%CPU	RSS	WCHAN	COMMAND
2448740	2448740	SNl	0.0	17664	futex_wait_queue	agent-leak-app-
2448740	2448841	SNl	0.0	17664	futex_wait_queue	agent-leak-app-
2448740	2448842	SNl	0.0	17664	futex_wait_queue	agent-leak-app-

전체 관제에서 워커 RSS는 17664 KiB = 17.250 MiB, 구간 CPU는 0.00%로 유지됐다. 프로세스는 s 상태였고 좀비 z가 아니었다. 주 스레드의 워커 완료 대기과 두 워커의 락 대기를 로그·TID 상태가 함께 뒷받침한다. 로그 이름과 개별 TID의 일대일 매핑은 별도로 확인하지 않았다.

UTC 관측 시각	console.log	agent_app.log	PID 상태
09:14:44.539	2881 bytes	1517 bytes	생존
09:14:49.892	2881 bytes	1517 bytes	생존
09:15:59.424	2881 bytes	1517 bytes	생존

09:16:03.856	2881 bytes	1517 bytes	정리 직전 생존
--------------	------------	------------	----------

로그 크기는 79.317초 동안 변하지 않았다. 마지막 앱 로그 시각 09:14:42.880부터 최종 기록까지는 약 80.976초다. `result.json`의 `alive_before_cleanup=true`와 종료 전 스냅샷이 지속 생존을 확인한다.



원문 자료:

- Before: [관제 로그](#), [CSV](#), [실행 로그](#), [ps:ps -L:top -H](#), [로그 크기](#), [종료 결과](#)
- After: [관제 로그](#), [CSV](#), [실행 로그](#), [스레드 상태](#), [로그 크기](#), [종료 결과](#)

3. Root Cause Analysis (원인 분석)

서로 반대 순서로 자원을 보유·요청한 것이 로그에서 확인된다.

```
Worker-Thread-1: Shared_Memory_A 보유 → Socket_Pool_B 대기
Worker-Thread-2: Socket_Pool_B 보유 → Shared_Memory_A 대기
순환: Thread-1 → Pool-B → Thread-2 → Memory-A → Thread-1
```

교착상태 조건	증거와 해석
상호 배제	Strict resource locking 안내와 LOCK ACQUIRED / BLOCKED 로그
점유 대기	한 자원을 Holding한 채 다른 자원을 요청
비선점	관측 구간에 상대 락을 빼앗거나 강제로 회수한 기록 없이 대기 지속
순환 대기	1은 2의 Pool-B, 2는 1의 Memory-A를 기다림

락 대기는 CPU를 소비하지 않고 스레드를 잠들게 할 수 있다. 그래서 PID와 RSS가 유지돼도 작업은 진행되지 않는다. `futex_wait_queue` 또는 낮은 CPU 하나만으로는 교착상태를 단정할 수 없지만, 이번에는 양방향 보유·대기 로그와 장시간의 정체가 함께 있어 순환 대기라는 설명을 뒷받침한다. 비선점 정책의 내부 구현과 락 객체 주소는 역공학하지 않았다.

TCP LISTEN은 유지되어도 작업 완료를 보장하지 않는다. 실제 네트워크 요청의 타임아웃은 별도로 측정하지 않았으므로 증상은 확인된 작업·로그 진행 중단으로 한정한다.

4. Workaround & Verification (조치 및 검증)

항목	Before	After
MULTI_THREAD_ENABLE	true	false
고정 변수	MEMORY_LIMIT=512, CPU=40%	동일
관찰 시간	90.021초	90.044초
워커 PID	2448740	2451259
최대 구간 CPU	0.00%	11.87%
RSS 추세	17.250 MiB로 정체	최대 517.676 MiB, 캐시 회수 후 재증가
마지막 로그	양쪽 WAITING / BLOCKED	MemoryWorker-CpuWorker 계속 진행
최종 콘솔 크기	2881 bytes	7747 bytes
관찰 후 정리	수집기 SIGTERM	수집기 SIGTERM

After에서는 All tasks completed, 부하 냉각, 18:18:17.146 Memory Cache Flushed, 18:18:43.359 Current Heap: 200MB까지 진행했다. WAITING/BLOCKED 로그는 없었다. PID 존재만이 아니라 실제 로그 진행과 메모리 회수로 관찰한 90초 동안 교착상태를 회피했음을 확인했다.

멀티스레드를 끄는 것은 동시성 감소를 감수하는 임시 회피다. 근본 개선은 일관된 락 획득 순서, 임계 구역 축소, 타임아웃 시 보유 락 해제, 자원 의존 구조 개선이다. 바이너리 수정은 수행하지 않았다.



[Analysis] 작업 로그를 통한 Round-Robin 스케줄링 추론

보너스 실측 분석 · 2026-09-18

1. 로그 관찰 개요

MEMORY_LIMIT=512, CPU_MAX_OCCUPY=40, MULTI_THREAD_ENABLE=false에서 앱이 Healthy System Monitoring을 선택했다. Task Scheduler가 A/B/C 작업을 등록하고 번갈아 실행하는 로그를 출력했다. [원문 실행 로그](#)를 분석했다.

2. 증거 자료

아래 시각은 KST다. 로그의 작업 이름·진행률·중단 문구를 기준으로 정리했다.

시각	작업	이벤트 / 진행률
18:12:52.682	A	Started / 20%
18:12:52.733	A	Calculating / 40%
18:12:52.785	A	Preempted / 40% 저장
18:12:52.837	B	Started / 20%
18:12:52.939	B	Preempted / 40% 저장
18:12:52.990	C	Started / 20%
18:12:53.093	C	Preempted / 40% 저장
18:12:53.144	A	Resumed / 60%
18:12:53.298	B	Resumed / 60%
18:12:53.451	C	Resumed / 60%
18:12:53.605	A	Resumed / 100%
18:12:53.656	B	Resumed / 100%
18:12:53.707	C	Resumed / 100%
18:12:53.758	Scheduler	All tasks completed

초기 작업 시작 간격 A→B는 155 ms, B→C는 153 ms, C→A 재개는 154 ms다. A의 첫 시작부터 중단 로그까지는 103 ms이며, 중단 로그부터 B 시작까지는 52 ms다. 전체 작업 로그 구간은 1.076초다. 이 시간은 로그 이벤트 간격이며 실제 커널 타임슬라이스 측정값은 아니다.

3. 패턴 분석 및 결론

하나의 작업이 완료되기 전에 다른 작업으로 넘어가고, A→B→C→A 순환과 저장된 진행률의 재개가 반복된다. 따라서 제시된 후보 중 앱 수준 Round-Robin 방식과 가장 잘 일치한다. 완료까지 한 작업만 수행하는 비선점 FCFS와는 맞지 않으며, 특정 작업이 우선권을 지속적으로 받는 패턴도 관측되지 않았다.

앱이 출력한 Preempted 문구만으로 Linux 커널이 SCHED_RR 정책으로 이 프로세스를 실행했다고 결론내리지는 않는다. sleep, 런타임, 앱 자체의 교육용 스케줄러가 동일 패턴을 만들 수 있다. 내부 소스나 커널 정책을 역공학하지 않았고, 이 결론의 범위는 앱 작업 로그다. 정책 구분은 [sched\(7\)](#)을 참고한다.

4. 장단점 및 적용

관점	분석
----	----

장점	여러 작업에 반복적인 진행 기회를 제공해 긴 작업의 독점을 줄임
단점	작업 교체 비용이 발생하며 할당 시간이 짧으면 오버헤드, 길면 응답 지연 증가
적합한 성격	작업 간 공정성과 빠른 초기 응답이 중요한 대화형 처리·요청 큐
비교 대상	배치 작업은 완료 처리량·캐시 효율을 더 중시할 수 있고, 긴급 작업은 우선순위 정책이 필요할 수 있음

이 관측에서는 공정한 교대 진행을 확인했지만 요청 지연·처리량을 벤치마크한 것은 아니다. 실제 서비스에 적용할 때는 대기 시간과 교체 비용을 따로 측정해야 한다.



[Bug] {장애 유형} - {한 줄 요약}

상태: 실측 전 초안 / 실측 완료 중 선택

1. Description (현상 설명)

- 발생 일시·시간대:
- 실행 계정, OS, CPU 수, 앱 SHA256:
- 재현 조건: MEMORY_LIMIT=..., CPU_MAX_OCCUPY=..., MULTI_THREAD_ENABLE=...
- 어떤 현상이 어떤 순서로 관측되었는가:
- 서비스 영향: 실제 확인한 영향과 추정한 영향을 구분한다.

2. Evidence & Logs (증거 자료)

재현 명령

```
# 실제 실행한 명령을 기록한다.
```

관제 수치

관측 시각	워커 PID	CPU (%)	RSS (MiB)	상태/스레드 수
측정 대기				

- metrics.csv, monitor.log, metadata.json, result.json 원문 링크:
- console.log와 앱 자체 로그 중 핵심 구간 발췌:
- ps / top / ps -L 출력과 관측 시각:
- 데드락인 경우 로그 마지막 수정 시각·크기와 관찰 지속 시간:

3. Root Cause Analysis (원인 분석)

- 관측된 사실:
- 증거에 근거한 추론:
- 관련 OS 동작 원리:
- 다른 가능한 원인과 배제 근거:
- 확인하지 못한 사항: 바이너리 내부 구현, 미측정 서비스 지연 등.

4. Workaround & Verification (조치 및 검증)

항목	Before	After
실행 증거 경로		
변경한 환경변수		
고정한 나머지 환경변수		
실제 관찰 시간		
종료 여부와 앱 종료 로그		
RSS 처음 → 최대		
최대 구간 CPU		
PID 생존 / 로그 진행 / 스레드 상태		

- 임시 조치와 그 효과:
- 관찰 시간 종료 시 생존한 경우 N 초 이상 생존이라고 기록한다.
- 수집기 종료 신호와 앱 자체 종료를 구분한다.
- 근본 해결 제안과 추가 검증: